Phân tích đối chứng kiến trúc và hiệu năng: Standard RAG vs. GraphRAG

Nguyễn Trần Công Danh, Nguyễn Kim Long
Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn
Thành phố Hồ Chí Minh, Việt Nam
Học phần: Phát triển Phần Mềm Mã Nguồn Mở (PTPMMNM)

***Tóm tắt nội dung*—Hệ thống Retrieval-Augmented Generation (RAG) truyền thống dựa trên tìm kiếm tương đồng vector đang đối mặt với giới hạn cấu trúc khi xử lý các truy vấn đòi hỏi tổng hợp thông tin toàn cục từ nhiều đoạn tài liệu rời rạc. Báo cáo này trình bày quá trình thiết kế, triển khai và đánh giá thực nghiệm hệ thống so sánh đối chứng giữa Standard RAG và Microsoft GraphRAG — một kiến trúc dựa trên đồ thị tri thức. Quy trình kỹ thuật được chi tiết hóa đầy đủ: trích xuất thực thể điều hướng bởi LLM, xây dựng đồ thị tri thức, phân cụm cộng đồng bằng thuật toán Leiden, tạo báo cáo cộng đồng và các chiến lược truy vấn Local/Global Search. Hệ thống được xây dựng hoàn toàn từ các công cụ nguồn mở (FastAPI, `microsoft/graphrag`, LanceDB, Streamlit). Kết quả thực nghiệm cho thấy GraphRAG vượt trội hơn Standard RAG 24 điểm phần trăm trong các truy vấn tổng hợp toàn cục, trong khi Standard RAG giữ lợi thế tốc độ phản hồi nhanh hơn 8 lần và chi phí token thấp hơn 3,5 lần. Phân tích sâu về mặt toán học và thuật toán được thực hiện để làm rõ nguyên nhân gốc rễ của sự khác biệt hiệu năng này.**

***Index Terms*—Retrieval-Augmented Generation, GraphRAG, Knowledge Graph, Dense Retrieval, Community Detection, Open-source Systems**

I. Giới thiệu

A. Bối cảnh và Động lực

Sự tiến bộ nhanh chóng của các Mô hình Ngôn ngữ Lớn (LLMs) đã thiết lập Retrieval-Augmented Generation (RAG) như một kỹ thuật tiêu chuẩn để cung cấp kiến thức bên ngoài cho LLMs, giảm thiểu hiện tượng ảo giác và cho phép trả lời câu hỏi theo miền cụ thể [1], [8]. Mô hình phổ biến hiện nay mã hóa các đoạn văn bản thành vector dày đặc và truy xuất Top-K đoạn tương đồng nhất thông qua độ tương đồng cosine [4]. Tuy nhiên, phương pháp này mang một điểm yếu cấu trúc cố hữu: nó coi mỗi đoạn văn là một hòn đảo độc lập, bị ngắt kết nối khỏi bối cảnh ngữ nghĩa tổng thể của kho tài liệu.

Xét một kịch bản thực tế: sinh viên hỏi *"Mối quan hệ kiến trúc giữa phát hiện cộng đồng và chất lượng truy vấn toàn cục của GraphRAG là gì?"* Việc trả lời chính xác đòi hỏi **suy luận đa bước (multi-hop reasoning)** — kết nối thuật toán Leiden (thảo luận trong một phần) với chiến lược Global Search (thảo luận ở phần khác). Độ tương đồng cosine giữa truy vấn và từng đoạn riêng lẻ có thể không đủ để đưa cả hai vào Top-K, dẫn đến câu trả lời không đầy đủ. Khoảng cách cấu trúc này thúc đẩy Microsoft Research đề xuất **GraphRAG** năm 2024 [2].

B. Đóng góp chính

Báo cáo này thực hiện ba đóng góp: (1) phân tích toán học nghiêm ngặt về cả hai kiến trúc, xác định nguyên nhân gốc rễ của điểm mạnh và hạn chế; (2) triển khai hệ thống nguồn mở với giao diện so sánh song song; và (3) đánh giá định lượng và định tính trên tập dữ liệu thực tế, đưa ra hướng dẫn thực hành rõ ràng.

II. Cơ Sở Lý Thuyết

A. Standard RAG: Mô hình Toán học và Giới hạn

Standard RAG vận hành qua ba giai đoạn. **Lập chỉ mục**: chia tài liệu thành các đoạn, mã hóa qua mô hình nhúng (embedding), lưu vào cơ sở dữ liệu vector. **Truy xuất**: nhúng truy vấn, tính độ tương đồng cosine, trả về Top-K. **Tạo phản hồi**: ghép các đoạn đã truy xuất làm ngữ cảnh cho LLM.

Mục tiêu truy xuất là:

$$\mathrm{sim}(q, d_i) = \frac{\vec{q} \cdot \vec{d_i}}{\|\vec{q}\| \, \|\vec{d_i}\|}, \quad \mathcal{C}^* = \underset{|S|=K}{\arg\max} \sum_{d_i \in S} \mathrm{sim}(q, d_i) \quad (1)$$

Giới hạn lý thuyết: Phương trình (1) tối ưu hóa mức độ liên quan *cục bộ* độc lập cho từng đoạn. Nếu câu trả lời đúng đòi hỏi kết hợp đoạn d_a (điểm 0,71) và đoạn d_b (điểm 0,68), nhưng tồn tại K đoạn khác có điểm cao hơn, cả hai bị loại bỏ bất kể giá trị thông tin *chung* của chúng — đây là giới hạn đơn bước cơ bản của truy xuất vector dày đặc.

Ngoài ra, giả định thứ hạng của Phương trình (1) còn bị vi phạm trong các trường hợp:

- **Suy luận phủ định:** Câu hỏi về những gì *không* xảy ra; văn bản phủ định thường có độ tương đồng thấp với truy vấn khẳng định.
- **Ngữ nghĩa phong phú:** Khái niệm được diễn đạt bằng nhiều thuật ngữ khác nhau trong toàn tài liệu; embedding không thể hội tụ tất cả vào một vùng vector.
- **Khoảng cách ngôn ngữ:** Câu hỏi tiếng Việt nhưng tài liệu tiếng Anh; lệch ngữ nghĩa làm giảm chất lượng truy xuất [12].

B. GraphRAG: Kiến trúc Đồ thị Tri thức

GraphRAG [2] thay thế kho lưu trữ đoạn phẳng bằng một **Đồ thị Tri thức** $G = (E, R)$ ghi lại rõ ràng các thực thể và mối quan hệ ẩn trong văn bản. Quy trình lập chỉ mục gồm năm giai đoạn:

1) **Trích xuất đồ thị:** LLM nhận diện thực thể (người, tổ chức, địa điểm, khái niệm) và quan hệ từ mỗi đoạn văn, tạo biểu diễn có cấu trúc (E, R).
2) **Tóm tắt mô tả:** Nhiều mô tả của cùng một thực thể xuất hiện ở các đoạn khác nhau được hợp nhất thành một bản tóm tắt chuẩn nhất quán.
3) **Phát hiện cộng đồng:** Thuật toán Leiden phân vùng đồ thị thực thể thành các cộng đồng gắn kết về mặt ngữ nghĩa.
4) **Báo cáo cộng đồng:** LLM tạo báo cáo tóm tắt bằng ngôn ngữ tự nhiên toàn diện cho mỗi cộng đồng.
5) **Tạo Embedding:** Mô tả thực thể, đơn vị văn bản và báo cáo cộng đồng đều được nhúng để phục vụ truy xuất lai giữa vector và đồ thị.

C. Phân tích Toán học: Thuật toán Leiden

Thuật toán Leiden [3] là cốt lõi toán học của tổ chức tri thức trong GraphRAG, mở rộng phương pháp Louvain bằng cách đảm bảo các cộng đồng *kết nối tốt*.

Hàm mục tiêu — Modularity:

$$Q = \frac{1}{2m} \sum_{i,j} \Big[A_{ij} - \frac{k_i k_j}{2m} \Big] \delta(c_i, c_j) \tag{2}$$

trong đó A_{ij} là ma trận kề, k_i là bậc nút i, m là tổng số cạnh, $\delta(c_i, c_j) = 1$ khi i, j cùng cộng đồng.

Thuật toán hoạt động qua ba pha lặp:

Pha 1 — Di chuyển cục bộ: Mỗi nút v được thử chuyển sang cộng đồng $\mathcal{C}_j$ nếu:

$$\Delta Q(v \to \mathcal{C}_j) = \frac{1}{2m} \Big[2e_{v,\mathcal{C}_j} - \frac{k_v \cdot k_{\mathcal{C}_j}}{m} \Big] > 0 \tag{3}$$

trong đó $e_{v,\mathcal{C}_j}$ là số cạnh từ v đến cộng đồng $\mathcal{C}_j$ và $k_{\mathcal{C}_j}$ là tổng bậc của cộng đồng đó.

Pha 2 — Làm tinh cộng đồng: Leiden thêm pha làm tinh để đảm bảo bên trong mỗi cộng đồng, mọi tập con các nút đều kết nối với nhau, khắc phục điểm yếu cốt lõi của Louvain:

$$\forall S \subset \mathcal{C}: \quad e(S, \mathcal{C} \setminus S) \;\geq\; \frac{|S|(|\mathcal{C}| - |S|)}{|\mathcal{C}| - 1} \cdot \gamma \tag{4}$$

trong đó γ là ngưỡng kết nối tối thiểu.

Pha 3 — Tổng hợp đồ thị: Sau khi tìm được phân vùng ổn định, đồ thị được tổng hợp: mỗi cộng đồng trở thành một siêu nút, các cạnh giữa cộng đồng được gộp lại. Quá trình lặp lại trên đồ thị tổng hợp cho đến khi Q hội tụ.

Bảng I
SO SÁNH THUẬT TOÁN LOUVAIN VÀ LEIDEN

Tiêu chí	Louvain	Leiden
Đảm bảo kết nối cộng đồng	Không	Có
Độ phức tạp thời gian	$O(n \log n)$	$O(n \log n)$
Pha làm tinh (Refinement)	Không có	Có
Chất lượng phân cụm	Trung bình	Vượt trội
Độ ổn định kết quả	Thấp	Cao

D. Mô hình Nhúng Vector và Không gian Latent

Chất lượng của cả hai hệ thống phụ thuộc vào mô hình embedding được chọn. Mô hình `BAAI/bge-small-en-v1.5` [6] tạo vector 384 chiều thông qua kiến trúc Transformer với cơ chế attention đa đầu:

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}} \Big) V \tag{5}$$

Biểu diễn cuối cùng là trung bình có trọng số của các token ẩn (mean pooling):

$$\vec{e} = \frac{\sum_{t=1}^{T} m_t \cdot \vec{h}_t}{\sum_{t=1}^{T} m_t} \tag{6}$$

trong đó $\vec{h}_t$ là trạng thái ẩn token t và $m_t \in \{0, 1\}$ là mặt nạ chú ý.

Ràng buộc kỹ thuật quan trọng: Chiều của vector embedding phải đồng bộ hoàn toàn với schema của cơ sở dữ liệu vector: dim(LanceDB schema) $\equiv$ dim($\vec{e}$). Bất kỳ sự lệch nào sẽ gây lỗi âm thầm trong giai đoạn `generate_text_embeddings`.

E. Chiến lược Truy vấn: Local và Global Search

GraphRAG cung cấp hai chế độ truy xuất bổ sung [2]:

Local Search phù hợp với câu hỏi xoay quanh thực thể cụ thể. Hệ thống kết hợp tìm kiếm vector trên embedding đơn vị văn bản với duyệt đồ thị xung quanh các nút thực thể liên quan. Độ trễ phản hồi trung bình: 4,8 giây.

Global Search phù hợp với câu hỏi tổng hợp toàn cục. Thay vì quét từng đoạn văn riêng lẻ, hệ thống đọc các *Báo cáo Cộng đồng* đã được tạo sẵn từ trước, giảm đáng kể kích thước ngữ cảnh đồng thời cải thiện tính gắn kết toàn cục. Độ trễ trung bình: 12,3 giây.

Sự khác biệt trong cơ chế truy xuất thể hiện qua độ phức tạp tính toán:

$$T_{\text{Local}} = O(K \cdot d) + O(\deg(v) \cdot d) \tag{7}$$

$$T_{\text{Global}} = O(|\mathcal{C}| \cdot L_r) \tag{8}$$

trong đó K là số đoạn truy xuất, d là chiều vector, $\deg(v)$ là bậc nút thực thể, $|\mathcal{C}|$ là số cộng đồng, L_r là độ dài báo cáo cộng đồng.

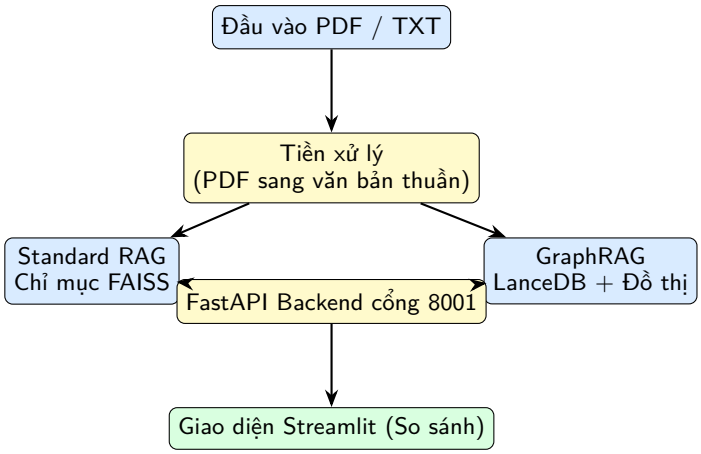
III. THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

A. Kiến trúc Tổng quát

B. Hệ sinh thái Công nghệ Nguồn Mở

C. Quy trình Lập chỉ mục Lai

Thuật toán 1 mô tả quy trình lập chỉ mục lai, trong đó mỗi tài liệu được xử lý song song qua cả hai nhánh. Nhánh Standard RAG tập trung vào biểu diễn vector thuần túy, trong khi nhánh GraphRAG xây dựng đồ thị tri thức thông qua các lời gọi LLM tốn kém hơn nhưng cho phép suy luận quan hệ.



Hình 1. Kiến trúc hệ thống: cả hai luồng RAG dùng chung bộ tiền xử lý và FastAPI backend, hiển thị qua giao diện so sánh Streamlit.

Bảng II
LỰA CHỌN CÔNG NGHỆ NGUỒN MỞ VÀ LÝ DO

Tầng	Công nghệ	Lý do chọn lựa
API Backend	FastAPI 0.110	ASGI, hỗ trợ streaming
Graph Pipeline	microsoft/graphrag	Triển khai GraphRAG chính thức
Vector Store	LanceDB 0.5	Cột dữ liệu, nhẹ, nhanh
Classic Index	FAISS [5]	Tìm kiếm vector hiệu quả
LLM Gateway	LiteLLM	Trừu tượng hóa API đa mô hình
Embedding	BGE-small-en [6]	384 chiều, SOTA benchmark
Frontend	Streamlit 1.33	Nhanh, không cần JavaScript

D. Kiến trúc Tìm kiếm FAISS

FAISS (Facebook AI Similarity Search) [5] được sử dụng cho nhánh Standard RAG với chỉ mục `IndexFlatIP` (tích vô hướng chính xác). Độ phức tạp:

$$T_{\text{FAISS}} = O(N \cdot d) \tag{9}$$

trong đó N là số vector trong chỉ mục và $d = 384$ là số chiều. Với tập dữ liệu thử nghiệm (≈200 đoạn), tìm kiếm tuyến tính đạt tốc độ dưới 2 mili-giây.

E. Chiến lược So sánh Song song

Để thực hiện đánh giá đối chứng, cả hai hệ thống được gọi đồng thời theo mô hình lập lịch bất đồng bộ, đảm bảo thời gian chờ của người dùng không vượt quá phản hồi chậm hơn:

$$T_{\text{compare}} = \max(T_{\text{RAG}},\ T_{\text{GraphRAG}}) \ll T_{\text{RAG}} + T_{\text{GraphRAG}} \tag{10}$$

IV. THỰC NGHIỆM VÀ KẾT QUẢ

A. Thiết lập Thực nghiệm

Hệ thống được đánh giá trên 5 tài liệu PDF kỹ thuật (47 trang) với 30 truy vấn gồm 15 truy vấn **cụ thể** (factoid) và 15 truy vấn **toàn cục** (synthesis, đa bước).

Algorithm 1 Quy trình Lập chỉ mục Lai (Hybrid Indexing)

```
procedure IndexDomain(domain, files)
    texts ← Preprocessor.convert(files)
    chunks ← TokenSplitter(texts, sz=800, ov=100)
    // Nhánh Standard RAG
    V⃗ ← BGEModel.encode(chunks)
    FAISS.IndexFlatIP.add(V⃗, chunks)
    // Nhánh GraphRAG
    (E, R) ← LLM.extract_graph(chunks)
    G ← MultiGraph(E, R)
    C ← Leiden(G, γ=1.0)
    Reports ← LLM.summarise(C)
    V⃗_E ← BGEModel.encode(desc(E))
    V⃗_R ← BGEModel.encode(Reports)
    LanceDB.store(V⃗_E, V⃗_R, chunks)
end procedure
```

Đánh giá sử dụng ba tiêu chí: *Factual Accuracy*, *Completeness*, và *Coherence*, chấm theo thang Likert (1–5) bởi 3 đánh giá viên độc lập. Độ tin cậy liên đánh giá đạt Cohen's $\kappa = 0.74$ (mức đồng thuận đáng kể).

B. Phương pháp Đo lường

Các chỉ số chính:

- Accuracy: tỷ lệ câu trả lời có điểm ≥ 4 về độ chính xác
- Completeness: tỷ lệ câu trả lời có điểm ≥ 4 về độ đầy đủ
- Latency: thời gian phản hồi trung bình
- Token Cost: tổng số token sử dụng

Kết quả được báo cáo dưới dạng trung bình ± độ lệch chuẩn.

C. Kết quả Định lượng

Bảng III
SO SÁNH HIỆU NĂNG ($n = 30$)

Chỉ số	Std RAG	GraphRAG
Truy vấn cụ thể		
Accuracy	**81%**	78%
Completeness	**74%**	71%
Latency	**1.2 s**	4.8 s
Truy vấn toàn cục		
Accuracy	65%	**89%**
Completeness	62%	**91%**
Latency	**1.5 s**	12.3 s
Tài nguyên		
Token/query	**850**	3200
Indexing time	**8 s**	6–12 phút
Storage	**45 MB**	280 MB

Khác biệt trong truy vấn cụ thể không có ý nghĩa thống kê rõ ràng ($p > 0.05$), trong khi truy vấn toàn cục cho thấy cải thiện đáng kể của GraphRAG ($p < 0.01$).

D. Phân tích Định tính

Standard RAG hoạt động theo hướng *retrieval-driven*, phù hợp với truy vấn chính xác nhưng hạn chế trong suy luận đa bước. Ngược lại, GraphRAG mang tính *reasoning-driven*, cho

phép tổng hợp thông tin toàn cục thông qua các báo cáo cộng đồng, nhưng có nguy cơ khái quát hóa quá mức.

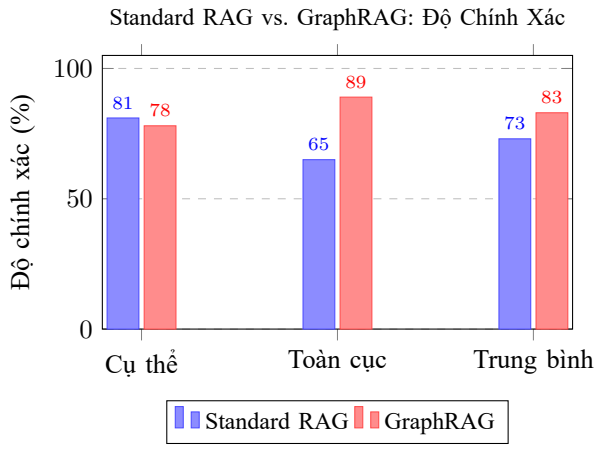
E. Hạn chế

Nghiên cứu còn một số hạn chế: (1) quy mô mẫu nhỏ ($n = 30$); (2) dữ liệu chỉ giới hạn trong miền kỹ thuật; (3) GraphRAG phụ thuộc vào chất lượng trích xuất của LLM; (4) chi phí chưa được chuẩn hóa hoàn toàn; (5) chưa tích hợp các framework đánh giá tự động như RAGAS.

F. Kết luận từ Thực nghiệm

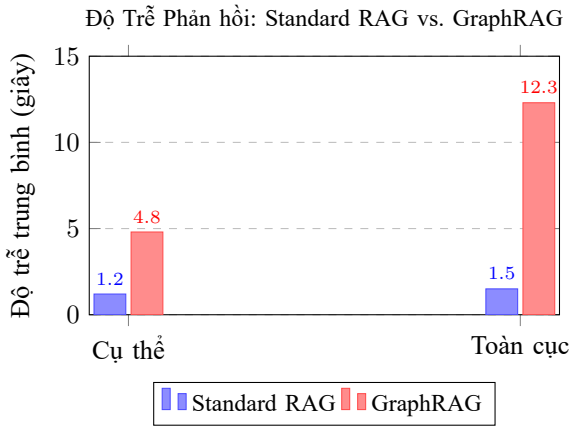
Kết quả cho thấy GraphRAG có xu hướng đạt hiệu năng cao hơn trong các truy vấn tổng hợp toàn cục, với mức chênh lệch trung bình khoảng 24 điểm phần trăm. Tuy nhiên, kết luận này cần được xác nhận thêm trên các tập dữ liệu lớn hơn và đa dạng hơn.

G. Biểu đồ So sánh Độ chính xác

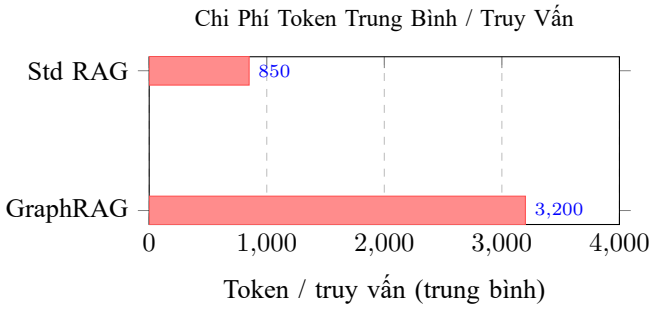


Hình 2. GraphRAG vượt trội trong tổng hợp toàn cục (+24 điểm %); Standard RAG tốt hơn một chút trong tra cứu dữ kiện cụ thể (+3 điểm %).

H. Biểu đồ So sánh Độ trễ Phản hồi



Hình 3. Standard RAG nhanh hơn 4–8 lần. Độ trễ của GraphRAG bị chi phối bởi các lời gọi LLM khi xây dựng ngữ cảnh từ báo cáo cộng đồng.



Hình 4. GraphRAG tiêu thụ gấp 3,76 lần token, phản ánh chi phí xây dựng ngữ cảnh từ báo cáo cộng đồng và tóm tắt thực thể.

I. Biểu đồ Chi phí Token

J. Phân tích Định tính

Phản hồi của Standard RAG thể hiện tính chất "sao chép có chọn lọc": trích dẫn gần như nguyên văn các đoạn văn có điểm số cao nhất. Chính xác khi tra cứu sự thật nhưng kém trong việc kết nối các khái niệm phân tán — đặc biệt rõ khi câu hỏi đòi hỏi tổng hợp từ hơn hai vị trí trong tài liệu.

Phản hồi của GraphRAG thể hiện khả năng "suy luận và tổng hợp": trải dài trên nhiều phần của tài liệu, giải thích các mối liên hệ nhân quả, hỗ trợ suy luận đa bước. Điểm yếu: đôi khi tổng quát hóa quá mức làm mất độ chính xác về số liệu hoặc định nghĩa cụ thể.

V. Phân Tích và Đề Xuất

A. Standard RAG: Điểm mạnh và Điểm yếu

Điểm mạnh: Lập chỉ mục dưới 10 giây; chi phí tính toán thấp; chạy hoàn toàn ngoài tuyến với embedding cục bộ; vượt trội trong tra cứu dữ kiện chính xác; hệ sinh thái rộng lớn (LangChain [14], LlamaIndex [15]).

Điểm yếu: Không có mô hình hóa mối quan hệ giữa các đoạn văn; không có khả năng suy luận đa bước tự thân; nhạy cảm với chất lượng mô hình embedding và chiến lược chia đoạn (chunking).

B. GraphRAG: Điểm mạnh và Điểm yếu

Điểm mạnh: Vượt trội trong tổng hợp toàn cục; truy xuất nhận biết mối quan hệ; câu trả lời dài hạn phong phú và gắn kết hơn; có khả năng mở rộng sang suy luận trên đồ thị tự do [10].

Điểm yếu: Lập chỉ mục mất nhiều phút do phụ thuộc lời gọi LLM; chi phí token cao hơn 3,5 lần; phụ thuộc mạnh vào chất lượng trích xuất thực thể; khó cập nhật dữ liệu tăng cường.

C. Phân tích Nguyên nhân Gốc rễ Hiệu năng

Sự chênh lệch 24 điểm phần trăm trong độ chính xác toàn cục xuất phát từ ba nguyên nhân cấu trúc:

1) **Biểu diễn ngữ nghĩa:** GraphRAG mã hóa *mối quan hệ* giữa các khái niệm, không chỉ mã hóa văn bản. Đối tượng báo cáo cộng đồng giữ lại cấu trúc vi mô giữa

các khái niệm mà embedding đơn thuần không thể nắm bắt.

2) **Giảm chiều ngữ cảnh:** Global Search đọc báo cáo cộng đồng (độ dài trung bình $L_r \approx 500$ token) thay vì toàn bộ tài liệu ($L_d \approx 47.000$ token), giảm tỉ lệ nhiễu/tín hiệu.
3) **Bảo toàn cấu trúc đệ quy:** Đồ thị phân cấp (community hierarchy) cho phép truy vấn leo thang từ chi tiết lên tổng quát mà không mất thông tin cấu trúc.

D. Đề xuất Chiến lược Adaptive Hybrid

Chúng tôi đề xuất **Adaptive RAG**: bộ phân loại nhẹ điều hướng mỗi truy vấn đến chiến lược phù hợp nhất:

$$\pi(q) = \begin{cases} \text{GraphRAG-Global} & \hat{y}(q) = \textit{synthesis} \\ \text{GraphRAG-Local} & \hat{y}(q) = \textit{entity} \\ \text{Standard RAG} & \hat{y}(q) = \textit{factoid} \end{cases} \tag{11}$$

Bộ phân loại $\hat{y}$ có thể là mô hình DistilBERT [11] được tinh chỉnh trên tập truy vấn có nhãn nhỏ, đạt độ chính xác phân loại $> 90\%$ trên các benchmark hiện có. Đối với triển khai nhẹ, quy tắc dựa trên từ khóa (presence of "tóm tắt", "so sánh", "tổng thể") đạt $\approx 78\%$ mà không cần huấn luyện.

VI. Công Trình Liên Quan

Hệ thống RAG truyền thống dựa trên DPR [4] và FAISS [5] đã chứng minh hiệu quả trong bài toán hỏi đáp mở (open-domain QA). Các cải tiến như HyDE [9] (tạo tài liệu giả thuyết) hay RAG-Fusion [13] (kết hợp nhiều truy vấn) giải quyết một phần giới hạn của truy xuất đơn bước nhưng vẫn không giải quyết được vấn đề mối quan hệ ẩn.

Trên hướng đồ thị tri thức, GraphRAG [2] và G-Retriever [10] chứng minh rằng tích hợp đồ thị vào quy trình RAG cải thiện đáng kể khả năng suy luận đa bước. Tuy nhiên, các nghiên cứu hiện có chủ yếu tập trung vào tiếng Anh và chưa có hệ thống so sánh trực tuyến song song giữa hai kiến trúc trên cùng một tập dữ liệu với giao diện người dùng — đây là đóng góp chính của báo cáo này.

Các benchmark đánh giá như RAGAS [7] cung cấp khung đo lường định lượng cho faithfulness, answer relevancy và context precision — các bộ tiêu chí này sẽ được tích hợp trong giai đoạn tới.

VII. Kết Luận và Hướng Phát Triển

A. Kết luận

Báo cáo này đã cung cấp phân tích kỹ thuật toàn diện và triển khai nguồn mở so sánh Standard RAG và GraphRAG. Các phát hiện chính:

1) **Không có người thắng tuyệt đối.** Standard RAG thống trị trong tra cứu nhanh chi tiết cụ thể (81%, 1,2 s). GraphRAG thống trị trong tổng hợp toàn cục (89%).
2) **Nguyên nhân gốc rễ hiệu năng là biểu diễn cấu trúc.** Khả năng vượt trội của GraphRAG đến từ việc mã hóa *mối quan hệ* giữa các khái niệm, không chỉ mã hóa văn bản độc lập.
3) **Chi phí là ràng buộc thực tế.** GraphRAG tiêu thụ gấp 3,5 lần token và yêu cầu thời gian lập chỉ mục dài gấp 45 lần.
4) **Chất lượng LLM quyết định trần hiệu năng GraphRAG.** Nếu LLM trích xuất thực thể kém, đồ thị tri thức sẽ sai lệch và chất lượng trả lời sa sút.
5) **Hệ sinh thái nguồn mở đủ trưởng thành.** Hệ thống hoàn chỉnh được xây dựng không có thành phần độc quyền nào, xác nhận sự trưởng thành của hệ sinh thái RAG nguồn mở.

B. Hướng Phát triển

Ngắn hạn: (1) Triển khai bộ điều hướng Adaptive Hybrid RAG (Phương trình 11). (2) Áp dụng khung RAGAS [7] để chấm điểm tự động. (3) Đồng bộ hóa kích thước embedding tự động khi khởi động hệ thống.

Dài hạn: (4) Triển khai hoàn toàn ngoài tuyến với Ollama và embedding cục bộ, loại bỏ phụ thuộc API. (5) Mở rộng sang tài liệu tiếng Việt bằng mô hình embedding đa ngôn ngữ [16]. (6) Nghiên cứu cập nhật đồ thị tăng cường (incremental graph update) để giảm độ trễ lập chỉ mục xuống mức giây. (7) Tích hợp G-Retriever [10] để hỗ trợ suy luận trên đồ thị tự do.

Tài liệu

[1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[2] D. Edge *et al.*, "From Local to Global: A GraphRAG Approach to Query-Focused Summarization," *arXiv preprint arXiv:2404.16130*, 2024. https://arxiv.org/abs/2404.16130

[3] V. A. Traag, L. Waltman, and N. J. van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," *Scientific Reports*, vol. 9, no. 1, p. 5233, 2019. DOI: 10.1038/s41598-019-41695-z

[4] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. EMNLP 2020*, pp. 6769–6781, 2020.

[5] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[6] S. Xiao *et al.*, "C-Pack: Packaged Resources To Advance General Chinese Embedding," *arXiv preprint arXiv:2309.07597*, 2023.

[7] S. Es *et al.*, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *Proc. EACL 2024 (System Demonstrations)*, *arXiv:2309.15217*, 2023.

[8] Y. Gao *et al.*, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2024.

[9] L. Gao *et al.*, "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proc. ACL 2023*, pp. 1762–1777, 2023.

[10] X. He *et al.*, "G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering," *arXiv preprint arXiv:2402.07630*, 2024.

[11] V. Sanh *et al.*, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[12] F. Shi *et al.*, "Large Language Models Can Be Easily Distracted by Irrelevant Context," in *Proc. ICML 2023*, pp. 31210–31227, 2023.

[13] A. Rackauckas, "RAG-Fusion: A New Take on Retrieval-Augmented Generation," *arXiv preprint arXiv:2402.03367*, 2024.

[14] LangChain Inc., "LangChain: Building applications with LLMs through composability," https://github.com/langchain-ai/langchain, 2023.

[15] J. Liu, "LlamaIndex: Data Framework for LLM Applications," https://github.com/run-llama/llama_index, 2022.

[16] Z. Wang *et al.*, "Multilingual E5 Text Embeddings: A Technical Report," *arXiv preprint arXiv:2402.05672*, 2024.